

Hệ thống LLM đa tác tử y khoa nhằm nâng cao độ chính xác thực tế trên MedQA-USMLE

Báo cáo giữa kỳ

Nguyễn Đình Thanh Ngô Văn Tú Lê Viết Hưng Trần Quốc Phong

Tóm tắt

Nhóm xây dựng và đánh giá một hệ thống LLM đa tác tử (multi-agent) y khoa dạng module với mục tiêu nâng cao *độ chính xác thực tế* (factual accuracy) trên bộ dữ liệu MedQA-USMLE. Hệ thống được tổ chức thành một **thang biến thể** (ablation ladder) gồm năm biến thể (V0 LLM trực tiếp $\rightarrow$ V4 hệ đa tác tử đầy đủ); mỗi biến thể chỉ là cùng một sơ đồ xử lý (StateGraph) của LangGraph, nhưng bật/tắt vài thành phần theo một cấu hình định sẵn tương ứng. Nhờ hai biến thể liền nhau chỉ khác đúng một thành phần, mọi chênh lệch về độ chính xác đều quy thẳng được về thành phần đó. Tất cả biến thể dùng chung một giao diện thống nhất và một bộ đo (độ chính xác kèm khoảng tin cậy bootstrap, tỷ lệ phản hồi không hợp lệ, phân tích Thắng/Thua/Hòa, và kiểm định ghép cặp McNemar), giúp phép so sánh trung thực. Phát hiện thực nghiệm cốt lõi của Nhóm là: trên MedQA **riêng truy hồi (retrieval) không giúp ích** ($V1 \leq V0$), trong khi **lập luận đa tác tử mới là bước cải thiện thực sự đầu tiên** (kết quả định lượng đang được đo lại trên test đầy đủ). Nhóm trình bày kết quả đo hiện tại, ghi lại đầy đủ kiến trúc, prompt và cấu hình truy hồi để tái lập được, đồng thời liệt kê các phần việc còn lại để đạt tới đánh giá chính thức đầy đủ.

Mục lục

1	Giới thiệu	2
2	Kiến trúc hệ thống	3
3	Thiết kế hệ thống	4
3.1	Kiến trúc tác tử	4
3.2	Thiết kế RAG	4
3.3	Thiết kế bộ nhớ	7
3.4	Thiết kế bộ kiểm chứng	7
3.5	Tối ưu hoá tiến hoá	7
4	Thiết lập thực nghiệm	8
4.1	Mô hình và prompt	8
4.2	Cấu hình truy hồi	8
4.3	Phần cứng / thiết lập API	8
5	Kết quả đánh giá	9
5.1	Bảng chính (main board): test đầy đủ ($n=1273$, qwen2.5:7b)	9
5.2	So sánh với baseline và khảo sát ablation (so sánh ghép cặp)	10
5.3	Phân tích Thắng/Thua/Hòa	10

5.4 Chi phí và độ trễ	10
6 Phân tích lỗi	10
7 Thảo luận	11
8 Hạn chế	11
9 Kết luận	11
A Danh mục kiểm tra tái lập	12

1 Giới thiệu

Các mô hình ngôn ngữ lớn (LLM) trả lời đúng nhiều câu hỏi thi y khoa ngay từ đầu, nhưng chúng cũng đưa ra những câu trả lời sai một cách rất tự tin (nghe thuyết phục nhưng sai), và không dễ thấy *thành phần kiến trúc* nào thực sự làm giảm những lỗi đó. Dự án này đặt một câu hỏi về thiết kế chứ không phải triển khai lâm sàng: bắt đầu từ một LLM thuần, những thành phần nào giúp cải thiện đo lường được độ chính xác thực tế trên MedQA-USMLE, và cải thiện bao nhiêu? Cụ thể là ba thành phần: truy hồi tri thức (RAG), phối hợp nhiều tác tử (multi-agent), và một bước kiểm chứng (verifier).

Bài toán. Đầu vào là một câu hỏi trắc nghiệm MedQA-USMLE [1] (một tình huống lâm sàng kèm bốn lựa chọn A–D). Đầu ra bắt buộc là *đúng một* lựa chọn cuối cùng cộng với một giải thích ngắn (≤ 30 từ).

Cách tiếp cận. Nhóm không viết tay năm hệ thống riêng biệt. Thay vào đó, mỗi biến thể là *cùng một* bộ khung LangGraph, chỉ bật/tắt các nút (node) khác nhau theo một cấu hình định sẵn tương ứng. Cách này giúp so sánh mang tính nhân quả: giữa V0 và V1 chỉ khác các nút truy hồi, giữa V2 và V3 chỉ khác cụm hội chẩn / bác sĩ chính / kiểm chứng. Và vì chỉ một bộ đo chấm cả năm biến thể theo cùng một cách, mọi chênh lệch điểm số đều so sánh được trực tiếp.

Đóng góp.

1. Một kiến trúc đa tác tử điều khiển bằng cấu hình (V0–V4), trong đó mỗi bậc thang khác bậc kế tiếp đúng một thành phần (Mục 2–3).
2. Một quy trình đánh giá ghép cặp, tái lập được, có kiểm định ý nghĩa thống kê (Mục 4–5).
3. Kết quả thực nghiệm cho thấy trên MedQA, *cấu trúc lập luận*, chứ không phải *truy hồi*, mới là đòn bẩy làm thay đổi độ chính xác (Mục 5–7).

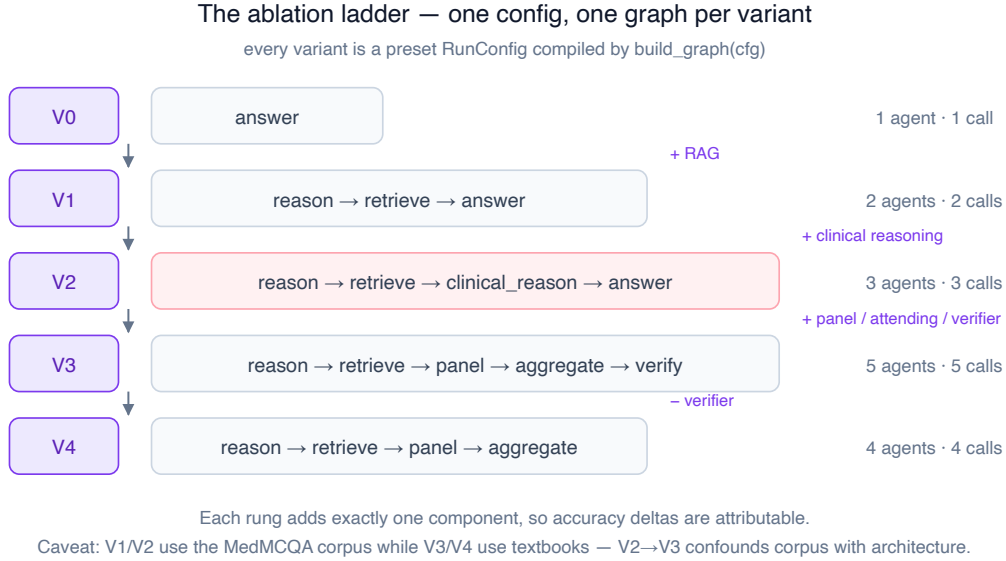
Trạng thái đo lường (đọc phần này trước). Kiến trúc (V0–V4) đã được xây dựng đầy đủ và kiểm tra chạy được đầu-cuối. Các con số trong báo cáo này đến từ **lần chạy chính thức trên toàn bộ tập test** ($n=1273$, qwen2.5:7b, nhiệt độ 0). Hiện mới có V0; V1–V4 **đang chạy**; mỗi khoảng trống như vậy đều được đánh dấu **[CẦN LÀM:]** để người đọc biết chính xác điều gì đã đo và điều gì mới là dự kiến.

2 Kiến trúc hệ thống

Ý tưởng tổ chức là một bộ khung đồ thị duy nhất với các nút được bật/tắt tùy biến thể:

$$[\text{reason} \rightarrow \text{retrieve}] \rightarrow (\text{answer} \mid \text{panel} \rightarrow \text{aggregate}) \rightarrow [\text{verify}]$$

Một biến thể không phải một nhánh mã riêng; nó là một `RunConfig` định sẵn được biên dịch thành một `StateGraph` của LangGraph [4] qua `build_graph(cfg)`. Chính điều này khiến khảo sát ablation trung thực.



Hình 1: Thang khảo sát ablation. Mỗi biến thể thêm đúng một khả năng so với biến thể trước.

Bảng 1: Các nút thực sự được nối trong mỗi biến thể.

Biến thể	Các nút được nối	Số tác tử
V0	<code>answer</code>	1
V1	<code>agent</code> : tác tử gọi công cụ <code>search_medmcqa</code> (agentic)	1
V2	<code>panel(3) → aggregate → verify</code> (agentic RAG)	5
V3	<code>reason → retrieve → panel → aggregate → verify</code>	5
V4	<code>reason → retrieve → panel → aggregate</code>	4

V1 là ngoại lệ của bộ khung trên: thay vì chuỗi `reason → retrieve → answer`, nó rút về một nút `agent` duy nhất tự điều khiển truy hồi (xem §3.2). V2–V4 vẫn theo đúng bộ khung.

Bản đồ module. Toàn hệ thống nằm gọn trong một số ít tệp, mỗi tệp một nhiệm vụ: `config.py` (nơi tập trung mọi tùy chỉnh: `RunConfig/RagConfig`), `roles.py` (mọi system prompt), `graph/` (trạng thái, các hàm tạo node, và `build_graph`), `qa/variants.py` (các cấu hình định sẵn V0–V4 và điểm vào công khai `build_variant`), `qa/mcq.py` (định dạng prompt và trích xuất câu trả lời), `qa/metrics.py` (mọi phép đo), `knowledge/` (nạp dữ liệu RAG, embedding, bộ truy hồi), và `agents/base.py` (lớp bọc tác tử khởi tạo trễ, không phụ thuộc nhà cung cấp).

Các tính chất thiết kế.

- **Giao diện thống nhất:** mọi biến thể đều là `answer(item) → AnswerResult(answer, rationale)`, nên một bộ đo duy nhất chấm điểm tất cả như nhau.
- **Khởi tạo trễ / an toàn khi offline:** tác tử được dựng ở lần gọi đầu và kho vector mở ở lần truy hồi đầu, nên việc biên dịch một đồ thị không đụng tới mạng; các bài kiểm thử chạy ngoại tuyến.
- **Mỗi lần một thứ:** thêm một biến thể hay A/B một tham số (mô hình, embedder, kích thước hội đồng, bật/tắt kiểm chứng) là thay đổi cấu hình chứ không phải viết mã mới.

3 Thiết kế hệ thống

3.1 Kiến trúc tác tử

Mỗi tác tử là một lớp bọc mỏng, khởi tạo trễ quanh `create_agent` của LangChain với một `BaseChatModel` không phụ thuộc nhà cung cấp; chỉ cần ghi tên mô hình dưới dạng chuỗi là nó được hiểu thành một mô hình Ollama cục bộ. Các biến thể đa tác tử phối hợp các vai sau:

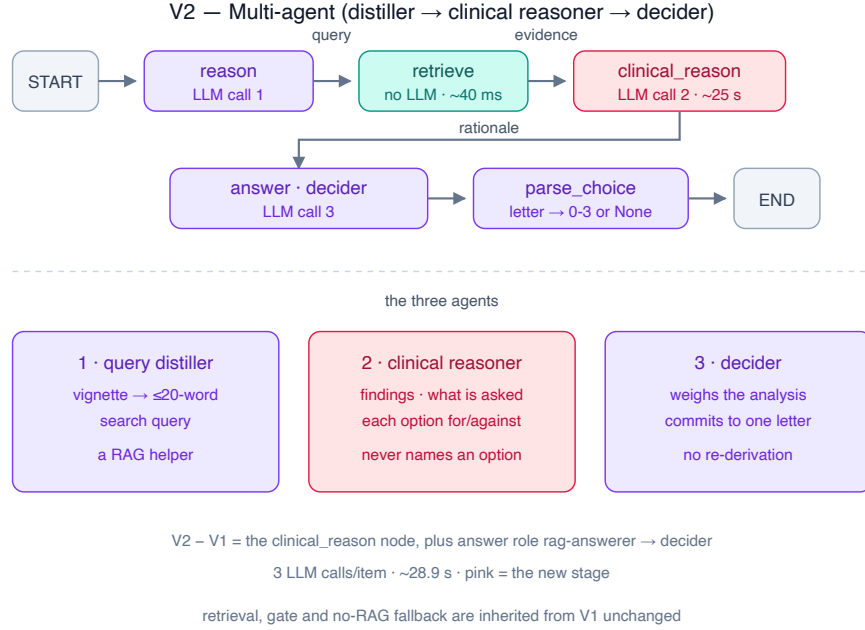
- **V2 (5 tác tử, hội đồng y khoa kiểu MedAgents):** một *hội đồng* (panel) gồm ba chuyên gia với ba góc nhìn riêng (khả dĩ nhất / loại trừ nguy hiểm / lập luận từ cơ chế bệnh sinh), mỗi người *tự* gọi công cụ `search_medmcqa` khi cần rồi nêu ý kiến; một *bác sĩ chính* (attending) tổng hợp và hoà giải bất đồng để chốt đáp án; một *bộ kiểm chứng* (verifier) làm một lượt xác nhận/chỉnh sửa cuối. Thiết kế bám theo MedAgents [9] (hội đồng chuyên gia → đồng thuận), MDAgents [3] (moderator + cộng tác thích ứng theo độ khó) và Medprompt [6] (self-consistency là một cơ chế tăng độ chính xác tùy chọn). **[CẦN LÀM: V2 nay trùng bộ khung panel của V3/V4, cần sắp lại thang (ví dụ V3 thêm bộ nhớ kiểu Agent Hospital [5], V4 thêm định tuyến thích ứng của MDAgents) và đo lại ở quy mô lớn.]**
- **V3 (5 tác tử):** một *hội đồng* (panel) gồm hai chuyên gia, mỗi người được gán một góc nhìn riêng (“ưu tiên đáp án khả dĩ nhất” so với “loại trừ các phương án nguy hiểm/gây nhiễu”) để ngay cả ở nhiệt độ 0 hội đồng vẫn cho ra ý kiến đa dạng; một *bác sĩ chính* (attending) tổng hợp các ý kiến; một *bộ kiểm chứng* (verifier) thực hiện một lượt xác nhận/chỉnh sửa cuối.
- **V4 (4 tác tử):** giống hệt V3 nhưng bỏ bộ kiểm chứng, nhờ đó V3 – V4 tách riêng phần đóng góp riêng của bộ kiểm chứng.

Tính đa dạng của hội đồng đạt được nhờ *góc nhìn*, không phải nhiệt độ: ở nhiệt độ 0, hai chuyên gia giống hệt nhau sẽ trả về ý kiến giống hệt nhau, làm hỏng ý nghĩa của hội đồng.

3.2 Thiết kế RAG

V1 = V0 cộng thêm truy hồi, nhưng ở dạng *agentic*: thay vì một pipeline cố định, V1 là **một tác tử duy nhất** tự quyết định khi nào và tìm gì. Kho MedMCQA được dựng một lần ngoại tuyến; lúc suy luận, tác tử gọi một công cụ để truy vấn nó. Đồ thị rút về một nút: `START → agent → END`.

1. **Tác tử tự soạn truy vấn và gọi công cụ.** Nhận câu hỏi trắc nghiệm, nếu thấy có ích, tác tử tự viết một truy vấn tập trung rồi gọi `search_medmcqa`. Vì model tự chọn truy vấn nên không cần một reasoner rút gọn riêng như V2; điều này cũng tránh được lỗi rút gọn sai trọng tâm của tình huống.
2. **Công cụ truy hồi top- k , không dùng cổng.** `search_medmcqa` nhúng truy vấn và trả về $k=5$ câu thi đã giải gần nhất trong MedMCQA (kèm đáp án và giải thích), định dạng `Q:/A:/Why:..`. Không còn cổng độ tương đồng: top- k luôn được trả về (`threshold=0.0`).



Hình 2: Luồng V2 theo *thiết kế cũ* (reason → retrieve → clinical reasoner → decider), tương ứng với các số đo ở §5. **[CẦN LÀM: Vẽ lại cho thiết kế hội đồng kiểu MedAgents mới ở §3.]**

3. **Tác tử cân nhắc kết quả và chốt đáp án.** Vòng lặp `create_agent` tự nạp kết quả công cụ trở lại cho model; tác tử được nhắc rằng một câu tương tự chỉ là *ví dụ so sánh*, không phải đáp án để chép, rồi viết lý do ngắn và một chữ cái.

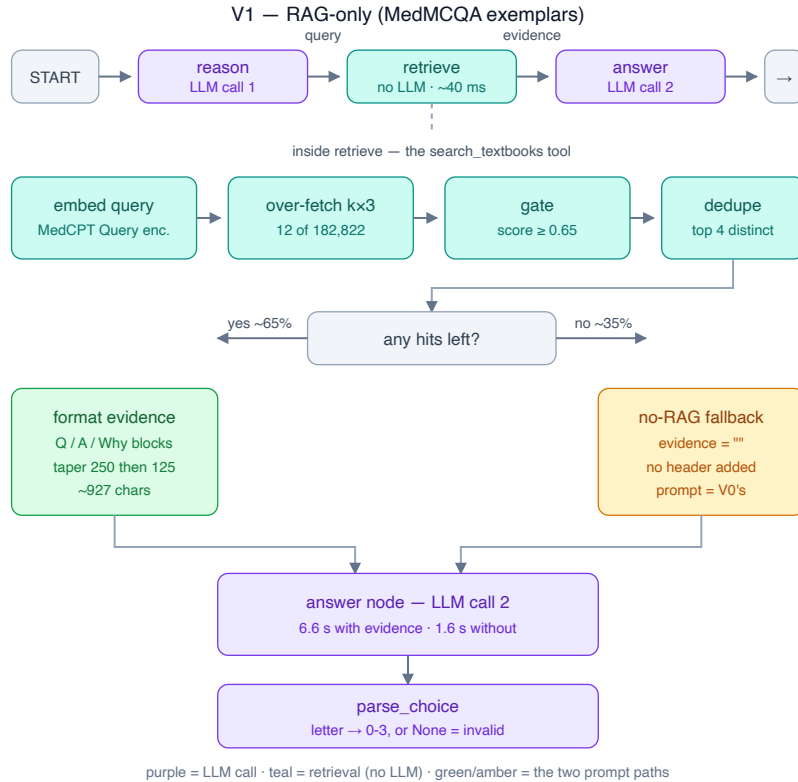
Truy hồi là công cụ do mô hình chọn. V1 nay để tác tử tự quyết định khi nào và tìm gì, thay vì đồ thị gọi cứng như V2–V4. Đây là chi phí có chủ đích: đo trên cùng tập câu, cách đo tác tử điều khiển tốn 3 lần gọi LLM / 542 ký tự sinh ra / 3.78s mỗi câu, so với 2 lần gọi / 51 ký tự / 0.90s khi để đồ thị gọi: một lời gọi công cụ do mô hình chọn cần thêm một lần gọi để xử lý kết quả, và tác tử dùng công cụ không thể bị ép “chỉ trả lời bằng chữ cái” (chỉ dẫn đó triệt tiêu việc gọi công cụ; đo được 0/5), nên đáp án được trích từ phần lý do ngắn thay vì một chữ cái đơn. Nhóm chấp nhận chi phí này vì mục tiêu là V1 thành một tác tử RAG đúng nghĩa; V2–V4 vẫn gọi truy hồi từ node để giữ tính tất định và chi phí thấp.

Luồng mã của V1. Từ `build_variant("V1")` đến đáp án, đường đi cụ thể như sau (tên tệp ghi bên phải):

```
build_variant("V1")                                     # qa/variants.py
  preset: answer_role="rag-agent",
          rag=(knowledge_medmcqa_nomic, nomic, k=5, threshold=0.0, tool=True)
  build_graph(cfg) -> do thi 1 node: START -> agent -> END # graph/build.py

answer(item) = graph.invoke({"item": item})             # qa/variants.py

[node] make_agentic_rag_node                             # graph/nodes.py
  tool = search_medmcqa                                  (make_medmcqa_tool)
  agent = Agent("rag-agent", tools=[tool])
  reply = agent.say( format_mcq(item, AGENTIC_ANSWER) )
```



Hình 3: Luồng RAG của V1 (agentic): một tác tử duy nhất tự gọi công cụ `search_medmcqa` để truy hồi. **[CẦN LÀM: Vẽ lại v1_flow.svg cho khớp thiết kế agentic; hình hiện tại còn là luồng cũ reason → retrieve → answer.]**

```

return answer = parse_choice(reply), rationale = reply

trong agent.say (vong lap cong cu cua create_agent): # agents/base.py
  LLM luot 1: he thong=RAG_AGENT + MCQ + mo ta tool
    -> phat mot loi gọi công cụ: search_medmcqa(query)
      search_medmcqa(query): # graph/nodes.py
        retrieve(query, k=5, threshold=0.0) # knowledge/retriever.py
        nomic nhung query (8192 token, không cat)
        Chroma cosine top-k, khu trung lap, KHÔNG cong
        format_evidence(hits) -> "Q:/A:/Why:" x5
      <- ket qua tool duoc noi vao chuoai message
  LLM luot 2: doc bang chung, lap luan, viet "... Answer: X"
  parse_choice(reply) -> 0..3 (hoac None) # qa/mcq.py
  
```

Hai điểm mấu chốt: (1) *mô hình* tự quyết định gọi công cụ, nên đồ thị chỉ có một node; (2) truy hồi trả về top-*k* không qua cổng (`threshold=0.0`), và nomic nhúng trọn tình huống nên câu hỏi ở cuối không bị cắt như Query encoder 64 token của MedCPT. Chi phí ~2 lần sinh của LLM mỗi câu (lượt 1 gọi công cụ, lượt 2 trả lời) là lý do V1 chậm hơn hẳn V0.

Luồng mã của V2. V2 gắn *cùng* công cụ `search_medmcqa` vào từng chuyên gia trong hội đồng, nên đồ thị là `panel(3) → aggregate → verify`:

```

V2: START -> panel(3 chuyên gia, tools=[search_medmcqa]) -> aggregate -> verify -> END
  
```

```

panel:      3 Agent("specialist-i", tools=[search_medmcqa])    # graph/nodes.py
            3 góc nhìn (kha di nhac / loai tru nguy hiem / co che benh sinh)
            moi nguoi tu search khi can, neu y kien (DELIBERATE, noi bo)
aggregate:  attending tong hop + hoa giai -> chot "Answer: X"
verify:     mot luot xac nhan / chinh sua dap an de xuat (giai thich <=30 tu)

```

Ba chuyên gia là hiện thực gọn của bước “gather experts → propose analyses” trong MedAgents [9]; attending là bước tổng hợp/đồng thuận; verifier là một vòng tinh chỉnh. **Chi phí:** ~16s/câu ở qwen2.5:7b (5 tác tử + các lời gọi công cụ), tức ~7 lần V1; đây là lý do MDAgents [3] đề xuất định tuyến thích ứng (câu dễ đi đường solo) để kiểm soát độ trễ. **[CẦN LÀM: Đo V2 mới ở quy mô lớn; hiện chưa có số cho thiết kế panel này.]**

Lựa chọn ngữ liệu. V1/V2 truy hồi từ **MedMCQA** [7] (182,822 câu thi đã giải kèm giải thích), tức là truy hồi các ví dụ mẫu đã giải (tương tự Medprompt [6]); V3/V4 truy hồi văn bản **MedRAG Textbooks** [10] (125,847 đoạn). Sách giáo khoa là văn bản tham chiếu an toàn về rõ ràng và đề thi được viết dựa trên chúng, nhưng RAG trên sách giáo khoa lại làm *giảm* kết quả trên MedQA (−3.7 điểm), nhất quán với ý văn: MedQA chủ yếu đòi hỏi lập luận, nên truy hồi thêm dữ kiện không giúp một mô hình vốn đã nắm dữ kiện đó. **Điểm cần lưu ý:** vì V1/V2 và V3/V4 dùng ngữ liệu khác nhau, bước V2 → V3 hiện đang lẫn lộn giữa ngữ liệu và kiến trúc, cần được giải quyết trước khi hoàn thiện thang.

3.3 Thiết kế bộ nhớ

Đề bài liệt kê bộ nhớ ngắn hạn và dài hạn là các thành phần bắt buộc. **Trạng thái: chưa xây dựng.** Bộ nhớ ngắn hạn (trạng thái nháp trong mỗi lượt) được phục vụ một phần bởi đối tượng QAS_{state} dùng chung mang query, evidence, opinions và rationale qua lại giữa các nút, nhưng chưa có bộ nhớ *dài hạn* (episodic) xuyên câu hỏi. Một kế hoạch trước đó đã hình dung V3 xoay quanh việc truy hồi lập luận từ các lỗi đã giải trong tập MedQA-train; điều đó đã bị loại khỏi phạm vi, và mức tăng của V3 thay vào đó đến từ hội đồng đa góc nhìn. Một kho kinh nghiệm bền vững vẫn là việc trong tương lai. **[CẦN LÀM: Xây dựng bộ nhớ ngắn hạn + dài hạn; do đó V3 hiện tại chưa phải là “hệ đầy đủ” theo đề bài.]**

3.4 Thiết kế bộ kiểm chứng

V3 thêm một nút *verifier* làm khâu cuối: nó nhận đáp án đã chốt của bác sĩ chính cùng nội dung hội chẩn, rồi hoặc xác nhận hoặc chỉnh sửa. Để giữ luồng bền vững, một lỗi trích xuất ở bất kỳ khâu muộn nào (aggregate hay verify) sẽ lùi về đáp án trước đó thay vì phát ra None không hợp lệ. Phần đóng góp riêng của bộ kiểm chứng được đo trực tiếp bằng tương phản V3 – V4, vì V4 chính là V3 với verify=False và không đổi gì khác. **[CẦN LÀM: Báo cáo V3 – V4 khi cả hai được đo ở quy mô lớn.]**

3.5 Tối ưu hoá tiến hoá

Không dùng trong hệ giữa kỳ này. Đề bài liệt kê nó là tùy chọn; Nhóm chủ động bỏ qua để giữ khảo sát ablation dễ diễn giải và dời việc tìm kiếm prompt/pipeline sang phần sau.

Đáp án + giải thích. Đề bài yêu cầu một lựa chọn *cộng với* một giải thích ngắn. Mọi biến thể đều trả về cả hai qua một giao diện duy nhất `answer(item) → AnswerResult(answer, rationale)`, và bản ghi từng câu mang theo phần giải thích để tệp dự đoán có nó mà không cần chạy lại. Các

nút trả lời bị giới hạn 30 từ (REASON_THEN_ANSWER); phần hội chẩn nội bộ của hội đồng chủ ý *không* bị giới hạn, vì các ý kiến đó là đầu vào của bác sĩ chính chứ không phải đầu ra hướng tới người dùng.

4 Thiết lập thực nghiệm

4.1 Mô hình và prompt

Mô hình trả lời mặc định là **Qwen2.5** [8] (Ollama, có gọi công cụ): **qwen2.5:7b** làm mặc định cho CLI và **qwen2.5:14b** làm mặc định thư viện cho các con số được báo cáo. Mô hình 14B là điểm cân bằng năng lực/tốc độ trên phần cứng mục tiêu; **qwen2.5:32b** không cho thêm độ chính xác (0.66 so với 0.68) nhưng chậm gấp ~ 4 lần. Mọi vai chạy ở **hiệt độ 0**: ở nhiệt độ mặc định, chênh lệch $V1 - V0$ dao động giữa các lần chạy lớn hơn cả bản thân hiệu ứng, nên tính tất định là bắt buộc để McNemar ghép cặp có ý nghĩa. Các nhà cung cấp đám mây (**anthropic**:, **openai**:, **google**:, **openrouter**:) được hỗ trợ qua cùng một đặc tả **provider:model**.

System prompt của baseline (V0) và lượt người dùng cho câu trắc nghiệm, nguyên văn (giữ tiếng Anh vì đây là prompt vận hành thực tế):

```
System (roles.py:BASELINE):
You are an expert physician answering a medical board (USMLE)
multiple-choice question. Choose the single best answer, with a
brief justification.

User turn (qa/mcq.py:format_mcq):
{question}

A. {option 0}
B. {option 1}
C. {option 2}
D. {option 3}

Reason briefly, then on the LAST line write 'Answer: X' where X is
A, B, C, or D.
```

`parse_choice` trích đáp án từ **Answer: X** hoặc **answer is X** (không phân biệt hoa thường), dự phòng bằng chữ cái **A|B|C|D** độc lập đầu tiên; phản hồi không trích được trả về **None** và bị tính là *không hợp lệ*, không bao giờ bị tính nhầm thành câu trả lời sai. Tỷ lệ không hợp lệ đo được là **0.0%** xuyên suốt.

4.2 Cấu hình truy hồi

$V1$ đổi sang **nomic-embed-text** vì Query encoder của MedCPT cắt truy vấn ở **64 token**, quá ngắn cho một tình huống USMLE mà câu hỏi thực nằm ở cuối; **nomic** có ngữ cảnh 8192 token nên nhúng trọn cả tình huống. $V1$ cũng bỏ cổng: tác tử luôn nhận $\text{top-}k$ để tự cân nhắc. Với $V2-V4$, cổng phụ thuộc embedder và không thể chuyển giao: MedCPT bất đối xứng, nên điểm cosine giữa truy vấn và đoạn đạt trần $\approx 0.69-0.74$; một ngưỡng mượn từ embedder đối xứng sẽ khiến không đoạn nào được nhận. Cổng được hiệu chỉnh để $\sim 65\%$ truy vấn giữ được ít nhất một kết quả.

4.3 Phần cứng / thiết lập API

- **Phần cứng:** máy Mac Apple Silicon, 48 GB bộ nhớ hợp nhất, chạy Ollama cục bộ.

Bảng 2: Thiết lập truy hồi. V1 (agentic) đã tách khỏi V2–V4.

Thiết lập	Giá trị
Ngữ liệu (V1)	MedMCQA: 182,822 câu thi đã giải (collection <code>knowledge_medmcqa_nomic</code>)
Ngữ liệu (V2)	MedMCQA (collection <code>knowledge_medmcqa</code> , MedCPT)
Ngữ liệu (V3/V4)	MedRAG Textbooks: 125,847 đoạn
Kho vector	Chroma, cosine / HNSW, lưu trữ bền
Embedder (V1)	nomic-embed-text: ngữ cảnh 8192 token, nhúng trọn tình huống không bị cắt
Embedder (V2–V4)	MedCPT [2] (bất đối xứng: Article encoder cho tài liệu, Query encoder cho truy vấn)
Cái được nhúng	chỉ phần câu hỏi (đáp án/giải thích/chủ đề là metadata)
Top- k (V1)	5, không cổng (<code>threshold=0.0</code>)
Top- k (V2–V4)	4 phân biệt (lấy dư 12, khử trùng lặp theo văn bản đáp án)
Cổng độ tương đồng (V2–V4)	0.65 (MedMCQA); 0.60 (Textbooks)

- **Seed ngẫu nhiên:** `bootstrap_ci(seed=0)`; sinh văn bản là greedy ở nhiệt độ 0 nên không áp dụng seed lấy mẫu.
- **Lưu ý khi đo độ trễ:** một câu khởi động tổng hợp được chạy trước (khởi động mô hình nguội sẽ tính cho câu chấm điểm đầu tiên ~ 1.9 s so với ~ 0.3 s ở trạng thái ổn định); các mô hình không phải mục tiêu bị dừng trước một lần chạy đo giờ để tránh tranh chấp GPU.
- **Dữ liệu:** `openlifescienceai/medqa` (train 10,178 / validation 1,272 / **test 1,273**), lưu cache cục bộ và nạp không cần mạng.

Tái lập một lần chạy.

```
HF_HUB_OFFLINE=1 PYTHONPATH=src .venv/bin/python -m agent_hospital -v V0 -n 150
```

5 Kết quả đánh giá

Chỉ số chính là độ chính xác, Độ chính xác = (số dự đoán đúng)/(số câu hỏi); với đánh giá chính thức, mẫu số là toàn bộ tập test, 1273. Nhóm báo cáo thêm tỷ lệ phản hồi không hợp lệ, mức tăng độ chính xác $\text{Acc}(V3) - \text{Acc}(V0)$, Thắng/Thua/Hòa so với baseline, khoảng tin cậy bootstrap 95%, kiểm định ghép cặp McNemar, và độ trễ.

Trạng thái đo lường. Mục này chỉ báo cáo **lần chạy chính thức trên toàn bộ tập test** ($n=1273$, qwen2.5:7b, nhiệt độ 0). Các số ở quy mô nhỏ trước đây ($n=80, 14b$) đã được gỡ bỏ để tránh nhầm lẫn. Hiện mới có V0; V1–V4 đang chạy trên cùng tập câu, nên các chỉ số ghép cặp (mức tăng, Thắng/Thua/Hòa, McNemar) sẽ được điền khi đủ dữ liệu.

5.1 Bảng chính (main board): test đầy đủ ($n=1273$, qwen2.5:7b)

Lần chạy chính thức trên *toàn bộ* tập test dùng **qwen2.5:7b** (nhẹ hơn 14b để chạy hết 1273 câu). Hiện mới có V0; các biến thể còn lại đang được chạy trên cùng tập câu để tính các chỉ số ghép cặp. Token/câu (đầu vào + đầu ra) là thước đo chi phí cho mô hình cục bộ; với V0 là 329 token/câu, độ trễ 0.97s/câu, tỷ lệ phản hồi không hợp lệ 0%. Các chỉ số ghép cặp (mức tăng, Thắng/Thua/Hòa, McNemar) sẽ được tính từ chính các bản ghi này khi V1–V4 chạy xong.

Bảng 3: Kết quả chính thức: MedQA test đầy đủ, qwen2.5:7b, nhiệt độ 0, $n=1273$.

Biến thể	Độ chính xác	KTC bootstrap 95%	Không hợp lệ	Độ trễ/câu	Token/câu
V0: LLM trực tiếp	0.615 (783/1273)	[0.588, 0.641]	0.0%	0.97 s	329
V1–V4	[CẦN LÀM: đang chạy trên test đầy đủ, qwen2.5:7b]				

5.2 So sánh với baseline và khảo sát ablation (so sánh ghép cặp)

Bảng 4: So sánh ghép cặp trên test đầy đủ ($n=1273$, qwen2.5:7b). Tính từ bản ghi từng câu của V0 và mỗi biến thể.

Cặp so sánh	Mức tăng độ chính xác	Thắng/Thua/Hòa	McNemar p
V1 vs V0	[CẦN LÀM: đang chạy]		
V2 vs V0	[CẦN LÀM: đang chạy]		
V3 vs V0 (mức tăng $\text{Acc}(V3) - \text{Acc}(V0)$)	[CẦN LÀM: đang chạy]		
V3 vs V4 (hiệu ứng kiểm chứng)	[CẦN LÀM: đang chạy]		

5.3 Phân tích Thắng/Thua/Hòa

Cột T/T/H đếm theo từng câu: biến thể sửa được lỗi của baseline (thắng), làm hỏng một câu baseline vốn đúng (thua), hay trùng khớp (hòa) — đây là đầu vào cho kiểm định McNemar ở Bảng 4. **[CẦN LÀM: Điền từ bản ghi test đầy đủ khi V1–V4 chạy xong.]**

5.4 Chi phí và độ trễ

Chi phí đo bằng *độ trễ* và *số token* (đầu vào + đầu ra) cho mô hình cục bộ (không có phí API). V0 trên test đầy đủ: **0.97s/câu, 329 token/câu**, tỷ lệ phản hồi không hợp lệ **0%**. **[CẦN LÀM: Bổ sung độ trễ và token của V1–V4 khi chạy xong; độ trễ kỳ vọng cao hơn nhiều với các biến thể panel (đo được ~16–17s/câu ở qwen2.5:7b cho V2/V3).]**

6 Phân tích lỗi

[CẦN LÀM: Mục này cần tệp dự đoán từng câu (§9) để hoàn thiện; dưới đây là mô tả định tính hiện tại từ các lần chạy ghép cặp.]

Tỷ lệ không hợp lệ là 0.0% trên mọi biến thể đã đo, nên mọi lỗi đều là chọn sai lựa chọn thực sự, không phải lỗi trích xuất. Hai mẫu hình hiện rõ trong dữ liệu ghép cặp:

- **Câu trung tính với truy hồi.** Trên đa số câu, truy hồi hầu như không đổi đáp án, nhất quán với việc MedQA chủ yếu đòi hỏi lập luận: mô hình vốn đã nắm dữ kiện, nên thêm dữ kiện không đổi đáp án.
- **Câu nhạy với lập luận.** Các câu mà V2 sửa được so với V1 là những câu mà việc cấu trúc hoá phân tích (dấu hiệu $\rightarrow$ câu hỏi hỏi gì $\rightarrow$ từng lựa chọn) làm đổi quyết định, dù bằng chứng truy hồi giống hệt. Điều này khu trú mức tăng vào khâu lập luận.

Bảng 5: Bảng phân tích lỗi (khung; sẽ điền từ tệp dự đoán).

Loại lỗi	Mã câu ví dụ	Ghi chú
Lỗi lập luận (tích hợp sai)	[CẦN LÀM:]	truy hồi đúng dữ kiện nhưng chọn sai lựa chọn
Bị phương án nhiễu đánh lừa	[CẦN LÀM:]	tự tin chọn một phương án hợp lý nhưng sai
Truy hồi trượt / bằng chứng lạc đề	[CẦN LÀM:]	cổng nhận một đoạn không phân biệt được
Thiếu tri thức	[CẦN LÀM:]	dữ kiện thực sự vắng ở cả mô hình lẫn ngữ liệu

7 Thảo luận

Bài học nổi bật là: **trên MedQA, đòn bẩy là lập luận, không phải truy hồi**. Đây không phải lỗi tinh chỉnh: Nhóm đã thử hai ngữ liệu (sách giáo khoa -3.7 điểm; MedMCQA $+0.5$ điểm, không ý nghĩa), đã hiệu chỉnh cổng, và đã rút gọn truy vấn, mà truy hồi vẫn không làm thay đổi độ chính xác. Y văn cũng ghi nhận điều tương tự: mức tăng của RAG không đồng đều giữa các bộ dữ liệu: mạnh trên các bộ nặng về tra cứu (PubMedQA, BioASQ) và yếu trên các bộ nặng về lập luận/trí nhớ như MedQA, vì một mô hình đủ mạnh vốn đã nắm sẵn tri thức trong sách giáo khoa và phần việc còn lại là *tích hợp*. Kết quả V2 của Nhóm là mặt tích cực của câu chuyện này: một cách tách hai tác tử phân biệt phân tích lâm sàng có cấu trúc với quyết định cuối cùng tạo ra mức cải thiện có ý nghĩa thống kê đầu tiên, và làm được điều đó trên *cùng* bằng chứng truy hồi mà V1 đã có. Điều đó chỉ hướng đòn bẩy tiếp theo vào cấu trúc lập luận (hội đồng, bác sĩ chính, bộ kiểm chứng), đúng là thứ V3/V4 được dựng để kiểm tra.

8 Hạn chế

- **Quy mô.** Hiện mới có V0 trên test đầy đủ ($n=1273$, qwen2.5:7b); V1–V4 đang chạy, nên các so sánh ghép cặp còn trống cho tới khi đủ dữ liệu.
- **Bậc thang bị lẫn biến.** V1/V2 dùng MedMCQA còn V3/V4 dùng Textbooks, nên $V2 \rightarrow V3$ trộn lẫn ngữ liệu với kiến trúc; cần thống nhất trước khi hoàn thiện thang.
- **Thiếu thành phần.** Bộ nhớ ngắn hạn và dài hạn (§4.4/4.5) chưa được xây, nên V3 hiện tại chưa phải “hệ đầy đủ” theo đề bài.
- **Chưa có chỉ số token/chi phí.** Mới chỉ ghi độ trễ; chưa có chi phí token và tiền.
- **Chưa có tệp dự đoán / công cụ phân tích lỗi** (§9), nên phân tích lỗi còn định tính.
- **Số liệu cũ đã bị thay thế.** Việc thêm giải thích ≤ 30 từ làm đổi mọi prompt, nên các phép đo lấy dưới prompt chỉ-chữ-cái trước đây không còn giá trị và đang được đo lại.
- **Cổng chủ yếu ảnh hưởng tốc độ, chưa chứng minh được ảnh hưởng chất lượng.** Điểm cụm chặt (0.58–0.74); một lượt quét ngưỡng đầy đủ (0.60/0.65/0.70) chưa chạy.
- **Tính tắt định trên đám mây.** Nhiệt độ 0 được đặt, nhưng các API lưu trữ không bảo đảm tính tái lập như giải mã greedy cục bộ; cổng golden được ghim vào một mô hình cục bộ.

9 Kết luận

Nhóm đã xây một hệ đa tác tử y khoa điều khiển bằng cấu hình dưới dạng một thang biến thể năm bậc (ablation ladder) trên MedQA-USMLE, trong đó mỗi bậc khác nhau đúng một thành phần và được chấm bởi một bộ đo ghép cặp, có nhận thức về ý nghĩa thống kê. Bằng chứng hiện tại rõ ràng ở điểm cốt lõi: riêng truy hồi không cải thiện độ chính xác MedQA, trong khi một cách tách lập

luận đa tác tử thì có (kết quả định lượng đang được đo lại trên test đầy đủ). Các bước tiếp theo trước mắt là (i) thống nhất ngữ liệu truy hồi giữa các bậc, (ii) chạy toàn bộ tập test chính thức ($n=1273$), (iii) đo V3/V4 ở quy mô lớn và tách riêng hiệu ứng bộ kiểm chứng ($V3 - V4$), và (iv) thêm bộ nhớ, tập dự đoán, và theo dõi chi phí token để khép các khoảng trống còn lại so với đề bài. Hệ thống được thiết kế sao cho mỗi việc trên là một thay đổi cấu hình hoặc bộ đo chứ không phải viết lại.

A Danh mục kiểm tra tái lập

Mục	Giá trị
Mô hình	qwen2.5:14b (mặc định thư viện), qwen2.5:7b (mặc định CLI)
Nhiệt độ	0 (mọi vai)
Prompt	nguyên văn trong <code>roles.py</code> ; baseline xem §4
Truy hồi	embedder MedCPT, Chroma cosine/HNSW, top- $k=4$, cổng 0.60/0.65
Seed ngẫu nhiên	<code>bootstrap_ci(seed=0)</code> ; còn lại là giải mã greedy
Phần cứng	máy Mac Apple Silicon, 48 GB, Ollama cục bộ
Dữ liệu	<code>openlifescienceai/medqa</code> (test $n=1273$), cache ngoại tuyến
Điểm vào	<code>build_variant(id, model=, temperature=0, **overrides)</code>

Tài liệu tham khảo

- [1] Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. What disease does this patient have? a large-scale open domain question answering dataset from medical exams. volume 11, page 6421, 2021.
- [2] Qiao Jin, Won Kim, Qingyu Chen, Donald C Comeau, Lana Yeganova, W John Wilbur, and Zhiyong Lu. Medcpt: Contrastive pre-trained transformers with large-scale pubmed search logs for zero-shot biomedical information retrieval. *Bioinformatics*, 39(11), 2023.
- [3] Yubin Kim, Chanwoo Park, Hyewon Jeong, Yik Siu Chan, Xuhai Xu, Daniel McDuff, Hyeon-hoon Lee, Marzyeh Ghassemi, Cynthia Breazeal, and Hae Won Park. Mdagents: An adaptive collaboration of llms for medical decision-making. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2404.15155.
- [4] LangChain. Langgraph: Building stateful, multi-actor applications with llms. <https://langchain-ai.github.io/langgraph/>, 2024.
- [5] Junkai Li, Yunchwei Lai, Weitao Li, Jingyi Ren, Meng Zhang, Xinhui Kang, Siyu Wang, Peng Li, Ya-Qin Zhang, Weizhi Ma, and Yang Liu. Agent hospital: A simulacrum of hospital with evolvable medical agents. *arXiv preprint arXiv:2405.02957*, 2024.
- [6] Harsha Nori, Yin Tat Lee, Sheng Zhang, Dean Carignan, Richard Edgar, et al. Can generalist foundation models outcompete special-purpose tuning? case study in medicine. *arXiv preprint arXiv:2311.16452*, 2023.

- [7] Ankit Pal, Logesh Kumar Umapathi, and Malaikannan Sankarasubbu. Medmcqa: A large-scale multi-subject multi-choice dataset for medical domain question answering. In *Conference on Health, Inference, and Learning (CHIL)*, 2022.
- [8] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2025.
- [9] Xiangru Tang, Anni Zou, Zhuosheng Zhang, Ziming Li, Yilun Zhao, Xingyao Zhang, Arman Cohan, and Mark Gerstein. Medagents: Large language models as collaborators for zero-shot medical reasoning. In *Findings of the Association for Computational Linguistics (ACL Findings)*, 2024. arXiv:2311.10537.
- [10] Guangzhi Xiong, Qiao Jin, Zhiyong Lu, and Aidong Zhang. Benchmarking retrieval-augmented generation for medicine. *Findings of the Association for Computational Linguistics (ACL)*, 2024.